



INTERNATIONAL UNIVERSITY OF GRAND-BASSAM
Excellence • Accountability • Opportunity

SCHOOL OF SCIENCE AND ENGINEERING

CSC 4305 · Software Engineering · Fall 2026

Agile Development II — XP, Kanban and the Limits of Agile

*The engineering half Scrum leaves out, the discipline of flow, and
the line above which none of it applies*

Session 6 of 28 · Friday 25 September 2026 · 10:00 – 11:20 AM

· Prof. Guillaume Sanje Mpacko, Ph.D.

Agile Development II — XP, Kanban and the Limits of Agile

SESSION 6 OF 28

Friday, 25 September 2026 · 10:00 – 11:20 AM · Unit 1 · Foundations and Process

LEARNING OBJECTIVES

- ✓ Explain what Scrum deliberately leaves out, and why a team that adopts only Scrum tends to slow down after three sprints.
- ✓ Name the nine core XP practices, say what each one is, and explain which other practice makes each one safe.
- ✓ Run the red–green–refactor loop of test-driven development on a real SANTE-BASSAM function.
- ✓ Define work in progress, cycle time and throughput, and use Little’s Law to relate them.
- ✓ Read a Kanban board, find the bottleneck from a flow log, and say what a WIP limit would change.
- ✓ Identify the parts of SANTE-BASSAM that must not be run in sprints, and say why.

FROM SESSION 5

Scrum gave your team a date — Friday 16 October — on which the clerk touches the software, and it will give you one every fortnight after that. We built the machinery that produces those dates: user stories with

acceptance criteria, points and velocity, three accountabilities, three artifacts, five events, and a written Definition of Done.

TEAM PROJECT · 0

first 15 minutes of class

Sprint 1 backlog — breaking S-04 and S-06 into tasks

Applying Session 5. The sprint 1 goal and the three stories are already agreed; layer 3 — the tasks — is what the team owes itself before Monday.

THE MODEL YOU ARE COPYING

S-05 was broken down for you in §5.3 of Session 5: six tasks, the conversation with the clerk first, the failing test second, the screen fourth. Open it and work to the same shape.

WHAT HAPPENS

Pair A

tasks for **S-04** — register a patient with the paper-card fields (5 pts)

Pair B

tasks for **S-06** — search results in under two seconds (3 pts)

Pair C

check both sets against the Definition of Done, and against S-05's tasks for anything duplicated

1. **Six minutes in pairs.** Each task is a verb and an object, small enough for one day, with **one named owner** and the test that proves it is finished.
2. **Six minutes reporting.** Pair C speaks last and says what is missing, not what is good.
3. **Three minutes to agree the board.** Every one of the six of you leaves owning at least one task, and no one owns tasks in only one story.

Output: the sprint 1 task list, with owners, in the progression log. That log is the accountability record for this project — if your name is not against a task, you have not started sprint 1.

Next session's slot: sprint 2 is planned in full, using the elicitation from Session 7.

1 · What Scrum leaves out

Scrum told you *when* the clerk sees the software. It said nothing whatsoever about how to write software that can still be changed on the fortnight after that, and the fortnight after that. Read the five events again:

not one of them mentions a test, a design or a line of code.

That silence is deliberate, and it has a predictable consequence. Here is what happens to a student team that adopts Scrum and nothing else:

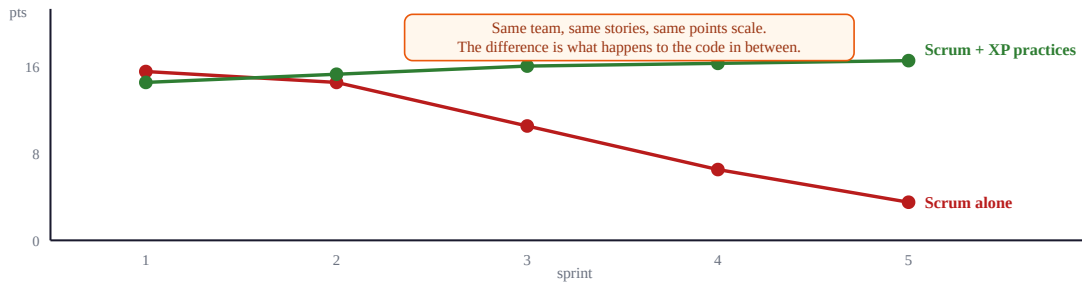


Figure 1. *The shape most student teams produce. Sprints 1 and 2 are fast because the code is new. From sprint 3 every change has to be made in code nobody tested, so each story costs more than the last, and velocity falls. The team concludes that agile does not work. What failed was the half Scrum never claimed to provide.*

TODAY IN ONE LINE

Scrum schedules feedback; **XP makes the code able to absorb it**; **Kanban** asks a different question altogether — not “what fits in two weeks?” but “why does work take so long to get through?”. And all three stop at a line, above which SANTE-BASSAM must not be run this way at all.

2 · Extreme Programming: the engineering half

Extreme Programming got its name from a simple idea: take the practices that everybody already agrees are good, and do them to an extreme degree. If code review is good, review every line as it is written — that is pair programming. If testing is good, write the test before the code. If integrating is good, integrate several times a day.

Nine practices. Read the definition column first; the third column is the one that matters most.

Test-driven development

TDD

Write a failing test for the next small piece of behaviour, write just enough code to pass it, then improve the code with the test still passing.

Made safe by: nothing — this is the practice everything else leans on. **It makes safe:** refactoring, collective ownership, simple design.

Continuous integration

CI

Every change is merged into the shared main branch within hours, and the whole test suite runs automatically on every push. A red build is fixed before anything else is done.

Made safe by: TDD — CI over an untested codebase just tells you faster that it still compiles.

Pair programming

Two people, one keyboard. One writes; the other thinks a step ahead and questions. They swap often.

Made safe by: TDD. Pairing without tests means two people confidently writing untested code, and twice the confidence in it.

Collective ownership

No module belongs to anyone. Whoever finds a defect fixes it, wherever it is.

Made safe by: TDD and CI — without them, editing someone else's module is a gamble, so people stop doing it and ownership silently returns.

Refactoring

Changing the internal structure of code without changing what it does, continuously, as part of ordinary work rather than as a project.

Made safe by: TDD. Refactoring without tests is not refactoring — you have no way of knowing whether behaviour changed.

Simple design

Build what today's stories need and no more. No abstraction added for a requirement that has not arrived.

Made safe by: refactoring. Simple design is only affordable if you are confident you can change the structure later, cheaply.

Small releases

Put something usable in front of the customer often — every sprint at least — even when the increment is one screen.

Made safe by: CI. Releasing often is only cheap if building and testing are automatic.

On-site customer

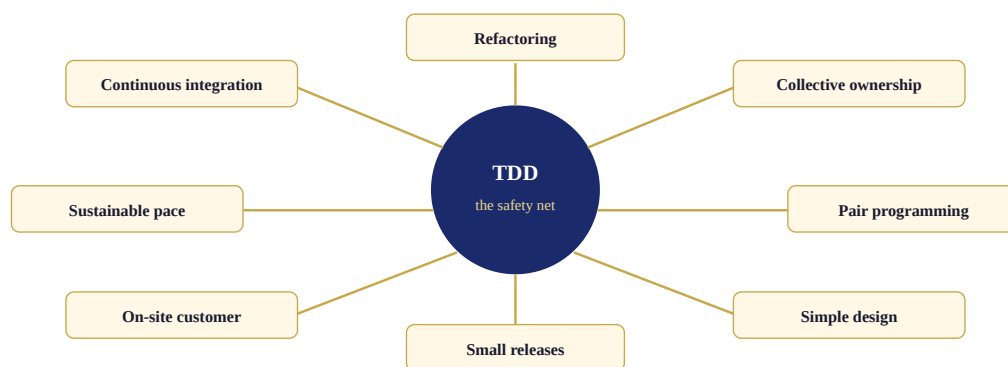
Someone who can answer questions about what is wanted is available continuously, not just at planning.

Made safe by: small releases — a customer with nothing to react to has nothing useful to say.

Sustainable pace

Plan for the hours the team can work indefinitely. No heroics before a review.

Made safe by: honest velocity — a team that plans beyond its measured velocity has already decided to work nights.



Remove the centre and every spoke becomes a liability. That is the whole argument for adopting them together.

Figure 2. Nine practices, and the one in the middle. Each practice on the rim is uncomfortable or dangerous on its own and safe in combination — and every path back to safety runs through the centre. This is why selective adoption tends to make things worse rather than partially better.

2.1 TDD worked, on a function you have already met

Story S-05 needs a ranking function: given a search term and a candidate patient, how good is the match? Acceptance criterion AC2 says a name misspelled by one letter must still be found. In TDD that criterion becomes a test *before* the function exists.

- 1 **RED — write the test that fails.** `rank("Kofi Modeste", "Kofy Modeste")` must return a score of at least 90. Run it. It fails, because `rank` does not exist yet. That failure is the point: it proves the test can detect the absence of the behaviour.
- 2 **GREEN — write the least code that passes.** Not the best code. Not the general code. The least. If a crude edit-distance check passes all three tests, that is today's implementation.
- 3 **REFACTOR — improve the structure, tests still green.** When story S-15 later needs a similarity score of its own, split `rank()` into `similarity()` and `rank()`. The existing tests do not change; if they still pass, behaviour is unchanged by construction.

WHY THE TEST IS WRITTEN FIRST, AND NOT JUST WRITTEN

A test written afterwards tests the code you happened to write. A test written first tests the behaviour you intended, and it is the only version of the requirement precise enough to run. In TDD the acceptance criteria stop being a document and start being an executable specification — and the traceability matrix in your Definition of Done becomes almost free to maintain.

If a model wrote the code

You will be using frontier language models to produce a good deal of the SANTE-BASSAM code, and that is a reasonable way to work. It does not weaken any of the nine practices — it sharpens three of them. **TDD:** the failing test is how you state the requirement to the model, and how you find out whether what came back actually does it. **Pair programming:** a generated function has been written by nobody, so it has been read by nobody — someone on this team reads it line by line before it merges. **Collective ownership:** whoever accepts generated code owns it afterwards, exactly as if they had typed it. “The model wrote it” is not a defect report, and it is not a defence.

APPLIED TO SANTE-BASSAM

The nine practices, made concrete for six students and one semester

TDD

AC2 of S-05 becomes the first failing test of sprint 1. The test is written at the desk, on the day of the paper-prototype session, while the clerk's wording is still fresh.

Continuous integration

GitHub Actions on every push, set up in sprint 0. Merging to `main` is blocked while the suite is red. Nobody merges red, ever, including at 23:00 the night before a review.

Pair programming

Six people make three pairs. Whoever went to the post on Friday pairs on Monday's changes with someone who did not, so what the clerk said reaches the code through two heads. Pairs are re-drawn each sprint.

Collective ownership

Whoever wrote the sync queue in sprint 0 may well be on the tablet interface in sprint 3. Anyone fixes a defect wherever they find it.

Refactoring

`rank()` splits into `similarity()` and `rank()` when S-15 needs the score.

Behaviour unchanged, structure improved, existing tests untouched.

Simple design

No user roles in sprint 1. One clerk, one laptop. Roles arrive in sprint 3, when the nurse needs a screen the clerk should not see — and not one sprint earlier.

Small releases

Installed at the post at every review, even when the increment is a single screen. The clerk keeps using the previous version in the meantime.

On-site customer

You cannot have one. The compromise: a fortnightly review plus a messaging channel where the Product Owner may ask the nurse one question a day, answered within 24 hours. The instructor answers when she cannot.

Sustainable pace

Sixteen points, not thirty — and not twenty-four just because there are six of you. Sixteen is what this team actually completed in sprint 0, carrying four other courses each. The review shows what is done, not what was survived.

Teams adopt the sociable practices — pairing, stand-ups — and skip the uncomfortable one, TDD. On SANTE-BASSAM the untested code is the sync path, and the consequence of a defect there is a lost health record for a real patient. Adopt all nine, or know precisely which safety net you have decided to work without, and write that decision down.

3 • Kanban and the discipline of flow

Scrum and XP both assume a timebox. Kanban removes it and asks a different question. Instead of “how much can we take on in two weeks?”, it asks “**why does one item take so long to get from start to finish?**” — and it usually turns out that the answer is not “because people are slow”. It is “because work is waiting”.

DEFINITIONS — THE FOUR YOU NEED

Work in progress (WIP) — the number of items that have been started and not finished.

WIP limit — a number written at the top of a board column. No item may enter that column while it holds that many. When a column is full, the correct action is not to start something else; it is to *go and help clear the full column*.

Cycle time — the elapsed time from an item entering the first active column to it reaching Done. Measured per item, then averaged.

Throughput — the number of items finished per unit of time. Not per person: per system.

LITTLE’S LAW

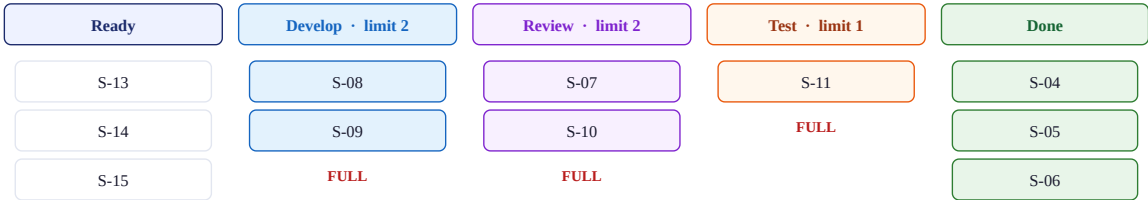
$$\text{average WIP} = \text{throughput} \times \text{average cycle time}$$

$L = \lambda W$ — true for any stable queueing system, regardless of how it works inside

Rearranged, it is the most useful sentence in this section: **cycle time = WIP ÷ throughput**. If your team finishes 1.5 stories a week and is carrying 6 started stories, each one takes 4 weeks to get through — no matter how hard anyone works. Starting a seventh makes every item slower. **The only way to finish sooner is to start less.**

WHY THIS MATTERS MORE TO A TEAM OF SIX THAN TO A TEAM OF FOUR

Six people have a natural urge to start six things, because nobody wants to look idle. Little’s Law says what that costs: with throughput fixed by how fast the team can *finish*, doubling the work in progress doubles every item’s cycle time and delivers nothing sooner. A Develop limit of two on a six-person board is therefore not a restriction, it is an instruction — it says **pair or triple up**, which is exactly what section 2 asked you to do anyway. The two practices are the same practice seen from two directions.



Nothing can move. Read the board before you read this.

A developer finishes S-08. Review is full, so S-08 cannot advance. She wants to pull S-13 from Ready — but Develop would then hold 2 again the moment S-08 leaves, and the queue in front of Review grows. **The correct move is to stop coding and go and review S-07.**

This is the whole idea: a WIP limit converts individual busyness into system throughput, by making waiting visible and starting expensive.

Figure 3. The SANTE-BASSAM board part-way through sprint 2. Every active column is at its limit. On a board without limits this same situation is invisible — everyone is busy, nothing is finishing, and nobody can say why.

3.1 Measuring flow — one story worked in full

Everything above is measurable from a **flow log**: the dates on which each story entered and left each column. Before the exercise asks you to do this for eight stories, here is the method on one of them, with the counting convention stated explicitly.

COUNTING CONVENTION — USE THIS ONE, SO THAT EVERYONE’S ANSWERS ARE COMPARABLE

- Count **working days only**, Monday to Friday. Number them from the start of week 5: Mon wk5 = day 1, Fri wk5 = day 5, Mon wk6 = day 6, and so on to Fri wk8 = day 20.
- Time in a column** = (day it left) – (day it entered). An item that entered Develop on day 1 and left on day 3 spent 3 days there: days 1, 2 and 3.
- Cycle time** = (day it reached Done) – (day it entered Develop) + 1.
- An item leaves one column on the same day it enters the next.

Worked example — S-04

Entered Develop

Left Develop

Left Review

Done

as dates	Mon wk5	Wed wk5	Mon wk6	Tue wk6
as day numbers	1	3	6	7
time in each column	Develop = 3 – 1 + 1 = 3 days		Review = 6 – 3 = 3 days	Test = 7 – 6 = 1 day
Cycle time for S-04 = 7 – 1 + 1				7 working days

Three days of coding, three days of waiting for a review, one day of testing. Note which of the three is the largest single opportunity, and note that nobody was lazy during it.

3.2 Kanban or Scrum?

	Scrum	Kanban
Unit of planning	The sprint — a fixed timebox	The single item — no timebox
Controls overload by	Committing only to the measured velocity	Refusing to start — WIP limits
Change mid-flight	Protected: the sprint goal is fixed for two weeks	Allowed: re-order the Ready column at any time
Key metric	Velocity, in points per sprint	Cycle time and throughput, per item
Suits	Work that can be batched behind a review date with a customer	A stream of arrivals of varying urgency — support, operations, maintenance
Costs you	The overhead of five events	The forcing function of a date — without a review Friday, nobody prepares

These are not rival religions

Plenty of teams put WIP limits on a Scrum board and keep the sprint. That combination is common enough to have a name, and there is nothing clever about it: the sprint supplies the date, and the limits stop six stories being half-finished on the morning of it.

4 • Where agile does not fit

Section 1 of Session 5 said agile is rational when feedback is cheap. The honest consequence is that there are conditions under which it is not, and you are expected to recognise them rather than apologise for

them.

No accessible customer

If nobody can answer questions between reviews, the feedback loop is broken and the Product Owner becomes a person guessing in a chair. Everything downstream inherits the guess.

Feedback is genuinely expensive

Spacecraft, implanted devices, anything that cannot be partially deployed. You cannot show a user half a satellite and ask what they think.

Regulatory evidence dominates

Certification requires traceable specifications written in advance. Agile teams can produce them, but must plan to — it does not fall out of the sprints.

Fixed-price, fixed-scope contract

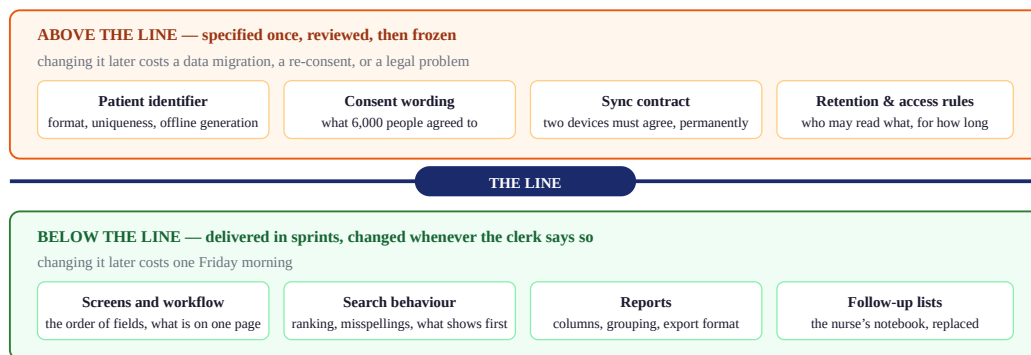
The commercial structure directly contradicts “responding to change”. Someone will lose the argument, and it will be in writing.

Interfaces others depend on

Iterate freely behind an interface. Never iterate the interface itself once a second party has built against it.

APPLIED TO SANTE-BASSAM

The line your Project Plan has to draw — and it is not a choice between two methods



Same project. Same team. Same fortnight. Two methods, because the cost of being wrong is different on each side.

Figure 4. *Your Project Plan does not choose between plan-driven and agile. It draws this line and defends where it put it. Knowing where the line goes is the engineering judgement this unit exists to teach.*

SESSION 6 TAKEAWAY

Scrum schedules the feedback; XP's nine interlocking practices make code that can absorb it, and every one of them is made safe by the tests at the centre. Kanban drops the timebox and attacks waiting instead: limit what you start, measure cycle time and throughput, and Little's Law does the rest. None of it applies above the line — the identifier, the consent, the contract — and a team that cannot say where its line is has not yet made an engineering decision.

5 · In-class exercise · where does the time go?

Twelve minutes. The flow log below is your own first four weeks. The counting convention is in §3.1 and S-04 is worked there — copy that method exactly.

Story	Entered Develop	Left Develop	Left Review	Done (test passed)
S-04	Mon wk5	Wed wk5	Mon wk6	Tue wk6 — <i>worked in §3.1</i>
S-05	Mon wk5	Fri wk5	Thu wk6	Fri wk6
S-06	Thu wk5	Fri wk5	Thu wk6	Fri wk6
S-07	Mon wk7	Wed wk7	Tue wk8	Wed wk8
S-08	Tue wk7	Thu wk7	Wed wk8	Thu wk8
S-09	Thu wk7	Fri wk7	Thu wk8	Fri wk8
S-15 pulled early	Mon wk8	Tue wk8	— still in Review	not done
S-16 pulled early	Tue wk8	Wed wk8	— still in Review	not done

In class — one story each, then the diagnosis (12 minutes)

1. **Six stories completed, six of you.** Take one each — S-04 is already done, so whoever takes it checks the worked answer and then helps a neighbour. Compute the three column times and the cycle time. **Four minutes.**
2. **Put all six cycle times on the board** and compute, together: the average cycle time, the throughput in stories per week over the four weeks, and — by Little's Law — the average work in progress the team was carrying. **Four minutes.**
3. **Which column holds work longest?** Give the number, not just the name. Then: S-15 and S-16 were pulled into Develop in week 8 by two people who had nothing to start, while three stories were still unfinished. What did that do to everything already in flight? **Four minutes.**

story	develop	review	test	cycle time
S-04	3	3	1	7
S-05				
...				
average cycle time = working days (five working days to the week)				
throughput	= stories / weeks = per week			
average WIP	= throughput × cycle time = stories			

Leave S-15 and S-16 out of the averages — they never finished — but do not forget about them in step 3.

6 · Homework · individual · due before Session 7 (Tuesday 29 September)

Yours alone again, roughly forty-five minutes, and again nothing hard. Every question continues something you started in class today — the arithmetic you did in §5, the practices in §2, the line in §4.

TEMPLATE

Download **CSC4305_HW06_Template.docx** from **CSC_4305_SWE-Assignments**, fill it in, rename it **CSC4305_HW06_FirstName_LastName.docx** and commit it to **/homework/S06/** in **your own private repository**. The team repository is for the project artifacts only.

A **.docx file**. Not a PDF, not a link. Full instructions are in the *Read this first* box at the top of the template.

- Q1 Your story, written up properly.** The one you took in class. Show the day numbers, the three column times, and the cycle time — with the arithmetic visible, in the shape of the S-04 table in §3.1.

The arithmetic is worth nothing; showing it is worth everything. A reader must be able to find your mistake.

- Q2** **Now do a second story** — any of the other five. Then one sentence: which of your two spent more time being *worked on* than *waited on*, and how you can tell.
-
- Q3** **Little’s Law, on your own.** Using the team figures from class, compute the average WIP. Then answer in two sentences: if the six of you each started one more story tomorrow, what happens to the cycle time of everything already in flight, and what happens to throughput?
The second half of that answer surprises people. Say it plainly.
-
- Q4** **Two practices, one safety net.** Pick any two of the nine XP practices in §2 where one makes the other safe. Say which way round it goes, and describe in three or four sentences what specifically goes wrong on SANTE-BASSAM if you adopt the second without the first.
Name the consequence in terms of a patient record, not in terms of “quality”.
-
- Q5** **Draw the line yourself.** Name one thing SANTE-BASSAM will need that was *not* in Figure 4 — anything at all. Say which side of the line it falls on and why, in two sentences. The test is the one from Session 5: what does it cost to change your mind about it in week 12?
-
- Q6** **Three sentences, your own opinion.** Should the team switch to Kanban for sprints 3 to 5? Weigh this: the review with the nurse is what makes the sprint boundary worth its overhead. Give one assumption you made, one trade-off you accepted, and one question you could not answer from the data.
There is no correct answer here. There are unsupported ones.
-
- Q7** **Reading.** Sommerville, *Software Engineering* 10th ed., §3.3–3.4. Optional and short: Beck, *Extreme Programming Explained*, chapters 1–3.
-

Key terms

Extreme Programming

test-driven development

red-green-refactor

continuous integration

pair programming

collective ownership

refactoring

simple design

small releases

sustainable pace

Kanban

work in progress

WIP limit

cycle time

throughput

Little’s Law

the line

Readings and references

- Sommerville, I. *Software Engineering*, 10th ed. — Chapter 3, §3.3–3.4.

- Beck, K. *Extreme Programming Explained: Embrace Change*, 2nd ed., 2004.
- Anderson, D. *Kanban: Successful Evolutionary Change for Your Technology Business*, 2010.
- Little, J. D. C. “A Proof for the Queuing Formula $L = \lambda W$ ”, *Operations Research*, 1961.
- Pressman, R. & Maxim, B. *Software Engineering: A Practitioner’s Approach* — Chapter 3.

International University of Grand-Bassam · School of Science and Engineering · **CSC 4305**, Fall 2026 · Unit 1 — Foundations
and Process

Next · Session 7 — Requirements Engineering I: Why Users Cannot Tell You, and Who to Ask